



SOFTWARE TESTING · UI QUALITY

GUI, Usability & Compatibility Testing

Three parts, one deck — how the interface looks, whether users can use it, and whether it works everywhere

Rebuilt from the FIT GUI & Usability Testing deck + reference checklists (Waghmare, Gerrard, Guru99, WalkMe; Rubin & Chisnell)
All GUI/usability screenshots are from the original course material; tool survey verified 2024–2026

What's inside

One combined deck covering three halves of interface testing.



Part 1 · GUI Testing

Testable elements · design heuristics (Nielsen, Norman, Shneiderman) · per-control checklists (text box, dropdown, button, checkbox, links, grids, dates, validation) · real screens · why it's hard · error taxonomy & strategy · AI.



Part 2 · Usability Testing

Six required components · test types & environments · the 9-part test plan · task & evaluator selection · performance vs preference metrics · questionnaires · analysis · a full tool survey with screenshots.



Part 3 · Compatibility

Browser × OS × device matrix · emulator vs simulator vs real device · the tool spectrum from offline visual regression → local emulation → online screenshot generation → cloud real-device simulation · AI.

PART 1

GUI Testing

What to test · per-control checklists · design heuristics · real examples · and AI. (Waghmare 2009, Gerrard/EuroSTAR, Guru99, WalkMe)

What is GUI testing?

Verifying the graphical interface behaves and looks as specified.

Checking every visible component — **buttons, icons, checkboxes, colors, menus, windows, dialogs** — against the specification.

The GUI decomposes into testable elements:

colors · fonts · alignment · orthography · inputs · buttons · links · symbols · pictures · videos · tables.



GUI Testing Tools — the element taxonomy

“Think as a user, not as a tester”

If the interface confuses users, they never come back — so the GUI must be right first.

GUI Testing Tools



“ Think as **USERS** ”



A button behind an overlay

A z-index regression hides “Sign in” — logins silently blocked.



NaN in a price field

An unhandled state renders “NaN d” — instant loss of trust.



Double-submit on double-click

No debounce → duplicate orders in the backend.

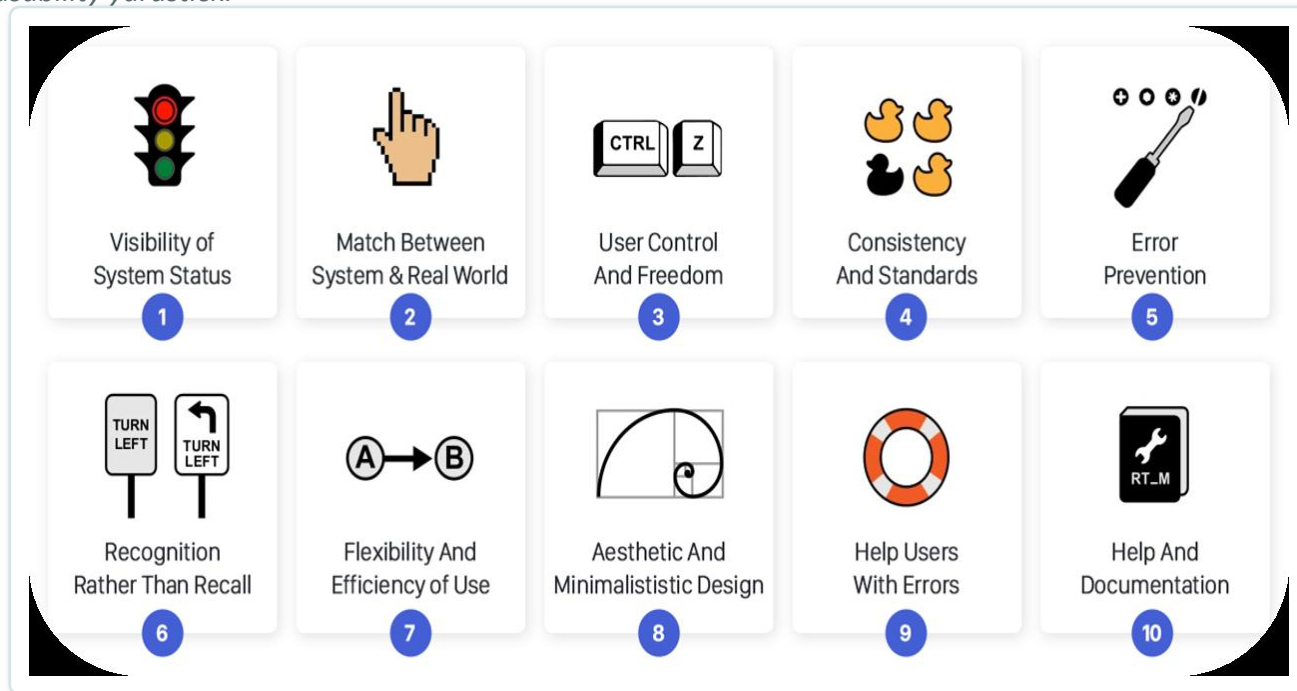


Broken layout on resize

Elements overlap/crop at another resolution.

Where checklists come from 1 — Nielsen's 10 heuristics

The most-cited usability yardstick.



Nielsen's 10 usability heuristics (course diagram)

Where checklists come from 2 — Norman's 6 principles

Each principle is a question the user silently asks the interface.

GUI Testing Tools

Norman's 6 Principles



VISIBILITY

Can I see it?



FEEDBACK

What is it doing now?



AFFORDANCE

How do I use it?



MAPPING

Where am I and where can I go?



CONSTRAINTS

Why can't I do that?



CONSISTENCY

I think I have seen this before.

Norman's 6 principles



Visibility

"Can I see it?"



Feedback

"What is it doing now?"



Affordance

"How do I use it?"



Mapping / Constraints / Consistency

"Where can I go? Why can't I? Seen this before?"

Where checklists come from 3 — Shneiderman's 8 Golden Rules

Shneiderman's Eight Golden Rules of Dialogue Design

1. Strive for consistency
2. Enable frequent users to use shortcuts
3. Offer informative feedback
4. Design dialogues to yield closure
5. Offer simple error handling
6. Permit easy reversal of actions
7. Support internal locus of control
8. Reduce short-term memory load

1. Strive for consistency
2. Enable frequent users to use shortcuts
3. Offer informative feedback
4. Design dialogues to yield closure
5. Offer simple error handling
6. Permit easy reversal of actions
7. Support internal locus of control
8. Reduce short-term memory load

Shneiderman's 8 Golden Rules of dialogue design

From heuristics to a review checklist

Turn the principles into a concrete, gradable GUI review sheet.

Inputs to a good GUI checklist:

- Requirements & UI design specs
- Nielsen 10 · Norman 6 · Shneiderman 8
- Platform guidelines & W3C WCAG
- Per-control checklists (next section)

Score each item Yes / No / N/A with a priority & severity — Mandatory vs Optional.

1	GUI Review Checklist						
2							
3	Project Code						
4	Version of the work product						
5	Reviewer (s)						
6	Review date						
7	Work product size						
8	Effort spent on review (man-hour)						
9							
10	Question	Yes	No	N/A	Note	Priority	Severity
11	Is graphical design of GUI reviewed or accepted by customer? If not					Mandatory	1
12	Has the waiver been stated?					Mandatory	1
13	Is one button set to be default?						4
14	Are the controls ordered according to normal behavior (Microsoft Style)?					Optional	4
15	Are all labels aligned uniformly?					Optional	4
16	Are text areas aligned uniformly?					Optional	4
17	Are there any spelling mistakes?					Mandatory	4
18	Are all non-enterable text areas of the same color (gray)?					Optional	4
19	Is the tab sequence correct?					Mandatory	4
20	Is cursor becomes hourglass when doing background processing?					Optional	3
21	Does cursor return to pointer after the task?					Optional	3
22	Do all fields match data length?					Optional	4
23	Do enterable fields accept entry only for the given lengths of String e.g. char (30)?					Optional	4
24	Are abbreviations used? If yes					Optional	4
25	Are they understandable for end users?					Optional	4
26	Are they consistently used across all screens?					Optional	4

A GUI Review Checklist (course example)

THE HEART OF GUI TESTING

Test every control

Each widget has its own checklist — text boxes, dropdowns, buttons, checkboxes, links, grids, dates. (Waghmare, 2009)

Checklist — Text Box

The most-used input; test state, limits, and every input class.

STATE & CONTENT

- Enable/disable per spec; disabled = greyed, no cursor, not editable
- Correct pre-population or blank; blinking cursor on focus
- Red asterisk on mandatory fields
- Stops at max width (test with capital W's); width per spec
- Rejects spaces-only; test spaces + chars

INPUT CLASSES & BEHAVIOR

- Test numbers, special chars (1 \$ # + -), alphanumeric + sign
- Formatted mask shown (SSN XXX-XX-XXXX); rejects short input
- Correct label; alignment consistent with siblings
- Scrollbar when content exceeds the box
- Double-click selects all; cursor changes over editable box

Checklist — Dropdown / Combo Box

SELECT-ONLY

- Arrow shown; default or blank value; can mandatory accept blank?
- Rejects values outside the list
- List non-empty; scrollbar for long lists
- Typing first letter selects the item
- Sorted (alphabetical default); names not truncated
- Selected item shows on top when reopened

EDITABLE COMBO

- Editable combo also: enter alphanumeric/special text
- Can delete selection; switch between text and list
- A typed value equal to a list value is not treated as different
- Pure dropdown: cannot type free text
- Alt-Down / Ctrl-F4 opens the list
- Blank/none sits at top or bottom consistently

Checklist — Button

Test the mouse path, the keyboard path, and the label.

BEHAVIOR & KEYBOARD

- Enable/disable; Tab reaches it in order; image changes on click
- Single / double / multi-click behave correctly
- Hover highlights + shows tooltip
- One default button (thick border) fires on Enter; Cancel on Esc
- Label meaningful, not abbreviated; “...” if it opens another screen

LABELS & CONVENTIONS

- Alt + underlined letter activates; Space activates focused button
- No duplicate hot-key letters on a screen
- Uncorrectable action → Yes/No confirmation
- All buttons similar size/shape, same font
- Cancel acts as Close when changes can't be undone

CONSISTENCY Search/Reset and Add/Delete must share size, font and spacing — colour signals intent (green = go, red = destructive).

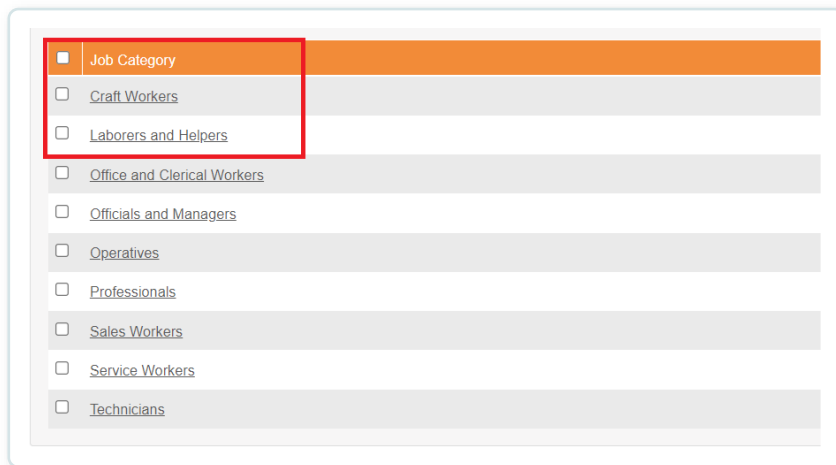
Checklist — Checkbox & Radio button

CHECKBOX

- Enable/disable; correct default checked/unchecked
- Appropriate label; Tab selects in series
- Mouse, Space, Enter, Alt+letter all toggle; clicking text toggles
- Correct events fire on change; alignment proper

RADIO / OPTIONS

- Correct default; only ONE selectable per group
- Arrow keys move the selection within the group
- Mouse, Space, Enter, Alt+letter change selection
- Handle a blank value from the DB for a fixed set



☐ Job Category
☐ Craft Workers
☐ Laborers and Helpers
☐ Office and Clerical Workers
☐ Officials and Managers
☐ Operatives
☐ Professionals
☐ Sales Workers
☐ Service Workers
☐ Technicians

Checkbox group — labels, defaults & alignment (course example)

Checklist — Hyperlink, Image, Grid, List, Date



Hyperlink

Standard color; underlined across every char; opens on click / Enter / Ctrl+click; Tab counts it; goes to the correct page.



Image

Not blurred; correct size; rotate/zoom if applicable; no broken images; properly aligned.



Grid control

Column + grid headings; header click sorts asc/desc with triangle; scroll via keys/wheel/buttons; double-click fires row event.



List box

Single select by mouse/arrows; letter jumps to match; double-click = select + open; scrollbar when needed.



Date field

Leap years; reject month 00/13, day 00/32, Feb 30; century change; correct entry format.



Browser button

Fills the linked textbox; hover colour change; Space/Enter/Alt trigger; disabled not clickable.

Checklist — input-field validation

Where most GUI defects actually live.

NUMERIC & ALPHA

- Numbers: lowest / highest / just-in-range / max / min / out-of-range above & below
- Handle blank in first & last position; both + and –; no division by zero
- Decide if negatives are allowed; include zero
- Alpha: blank/non-blank; invalid chars & symbols; valid chars

RULES & MESSAGES

- Every failed rule → sensible message; user must fix before proceeding
- Mandatory where the DB column is NOT NULL; asterisk shown
- Char limit = DB column size; pop-up “limited to N chars” at max
- No error for optional blank fields; test every field with special chars

GOLDEN RULE On error, return focus to the offending field and highlight it — even if the user clicked OK without tabbing off.

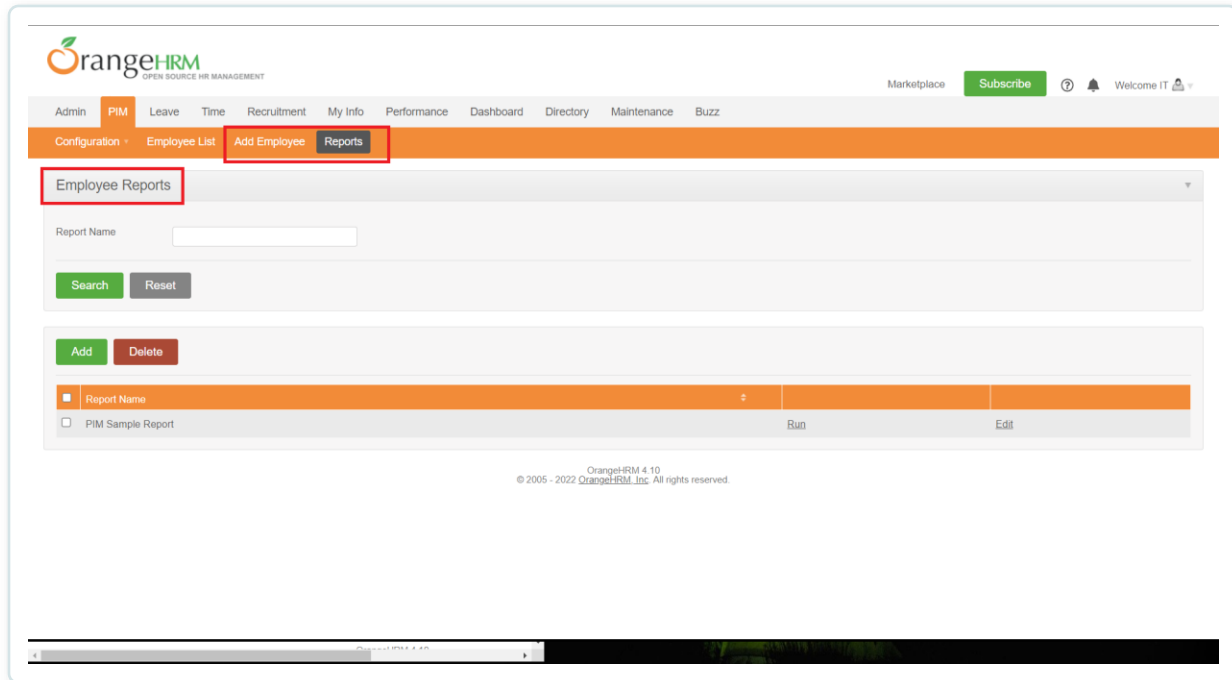
REAL SCREENS

Examples from real apps

The same checklist applied to production interfaces — spot the issues.

Example — consistency & layout (OrangeHRM)

Fonts, alignment, button grouping, and control state on a real admin screen.



OrangeHRM — System Users (annotations mark checklist points)



Check here

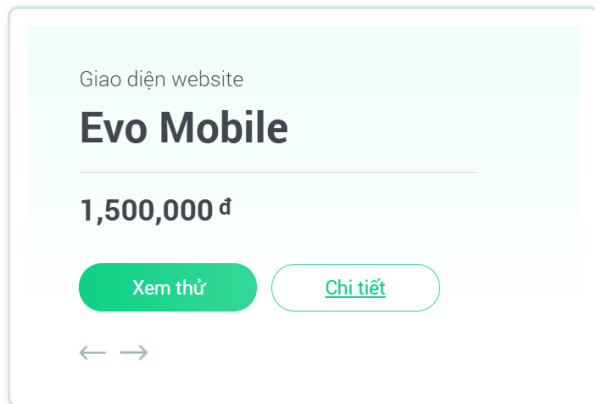
Consistent header font & color; table columns aligned; Search/Reset grouped and equal-sized; disabled rows greyed; pagination present.



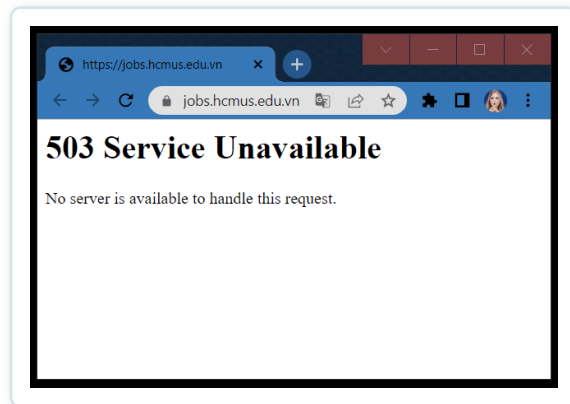
Then ask

Does the layout hold at 1366×768 and 1920×1080? Are actions reachable by keyboard?

Example — buttons, links & error pages



Evo Mobile — “Xem thử” (primary) vs “Chi tiết” (secondary)



503 error — is the message clear & recoverable?

WHAT TO VERIFY Primary vs secondary buttons are visually distinct · links styled & working · error pages are human-readable, on-brand, and offer a way back — not a raw server dump.

Why GUI testing is hard

The GUI is an event-driven world — the input domain is practically infinite. (Gerrard)



Event-driven

The user can click any pixel, in any order — no single control flow.



Unsolicited events

OS events (battery, network, notifications) interrupt anywhere.



Hidden dependencies

Behind one click sit many rules and object/window state.



“Infinite” input domain

Field-entry order alone gives $n!$ sequences; clicks, shortcuts, devices.



Many ways in / out

Menu, toolbar, shortcut, context menu — all must behave the same.



Window management

Move, resize, maximize, minimize, close — each can corrupt layout/state.

GUI error taxonomy + four-stage strategy

COMMON GUI ERROR TYPES

- Data validation; wrong field defaults
- Mandatory fields not enforced
- Queries retrieve wrong fields / multiple rows
- Stale on-screen data; window-object ↔ DB mismatch
- Wrong window modality; broken system commands
- Control/menu state not aligned with data state

FOUR TEST STAGES

- 1 · Low level — checklist & navigation testing
- 2 · Application — EP, boundary values, decision tables, state-transition
- 3 · Integration — desktop, client/server, synchronization
- 4 · Non-functional — soak, compatibility, platform

Automation — automate the right things

Pareto: ~80% of the benefit comes from ~20% of the tests.



Automate

Navigation & checklist testing · high-volume EP/BVA/decision-table cases · soak, compatibility, platform runs.



Keep manual

Synchronization and complex-interaction tests; anything highly sensitive to UI change.



Coded > recorded

Hand-coded scripts (Page Object Model) survive UI change; recorded scripts are fragile.

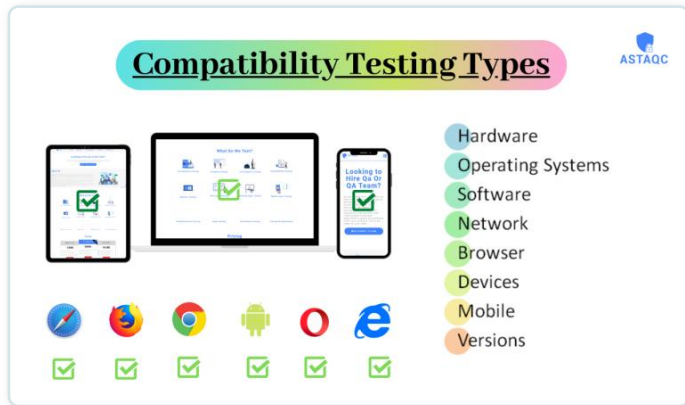


Hybrid is normal

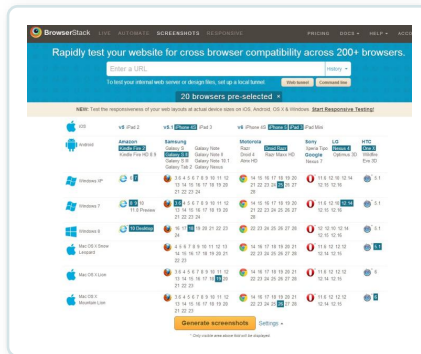
Mix automated regression with exploratory manual testing — neither alone is enough.

GUI across browsers & devices

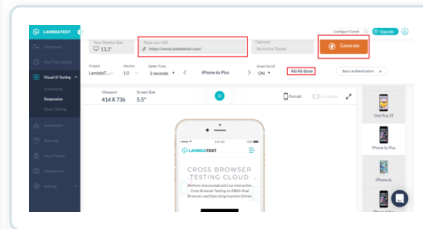
The same GUI must render on every browser × OS × device.



Compatibility testing dimensions (course diagram)



BrowserStack



LambdaTest

HOW Build the matrix from real analytics; test P1 combos every release. Cloud device farms (BrowserStack, LambdaTest, CrossBrowserTesting) run one test across thousands of real browsers.

STATE OF THE ART

AI in GUI testing

Visual AI, browser agents and self-healing — how GUI testing is changing in 2024–2026.

AI 1 — Visual AI & self-healing locators



Visual AI (Applitools / Percy)

ML compares rendered screens, ignoring acceptable render noise while catching real layout drift — far fewer false positives than pixel diff.



Self-healing locators

When the primary selector breaks, the tool re-resolves the element (rule-based ~40–70%, LLM intent-based 75–90%+). Log every heal.



Natural-language tests

testRigor / mabl author tests in plain English — resilient because there are no hard-coded locators.



Watch out

Self-healing can mask a real regression; keep deterministic assertions on critical flows.

AI 2 — browser agents & AI-generated UIs



Browser agents (Playwright MCP)

An LLM drives a real browser via the accessibility tree (roles + names, not pixels) — explores the GUI from high-level intent. Free & official.



Testing AI-generated UIs

v0, Lovable, Bolt, Cursor churn the DOM and vary between runs. Use role/label locators + visual-AI baselines; treat generated code as untrusted.



Non-deterministic output

For LLM-backed screens, assert on structure/schema, not exact text; add accessibility scans (axe-core) every regeneration.



Keep humans in the loop

AI agents are non-deterministic — great for exploration, weak as a regression gate. Review generated components.

Tools & free stack for GUI testing

Need	Tools	Free?
E2E GUI automation	Playwright · Selenium · Cypress	Free (open source)
Visual regression AI	Applitools Eyes · Percy · BackstopJS	Free tier / OSS
Cross-browser device farm	BrowserStack · LambdaTest · CrossBrowserTesting	Free trials
Accessibility	axe DevTools · WAVE · Lighthouse	Free
Agentic GUI testing	Playwright MCP + an LLM client	Free (open source)
Natural-language tests	testRigor · mabl	Free tier (OSS tests)
Checklists	Waghmare per-control · WCAG 2.2 · Front-End Checklist	Free

EXERCISE

Your turn: run a real GUI review

Per-control audit

Take the OrangeHRM / MentorUS login & a form. Apply the text-box, button, dropdown & validation checklists — log every deviation with priority & severity.

Heuristics + a11y

Do a heuristic walkthrough (Nielsen 10) and run axe-core / Lighthouse. Compare what the checklist caught vs what the tool caught.

AI vs manual

Drive the same screen with Playwright MCP + an LLM; add one AppliTools/Percy visual checkpoint. What did AI find that your checklist missed — and vice versa?

Deliverable: a filled GUI review sheet + a one-page note comparing manual, checklist and AI findings.

PART 2

Usability Testing

Method, planning, tasks, metrics & tools — based on Rubin & Chisnell, Handbook of Usability Testing.

What is usability testing?

A process that employs representative users to evaluate a product against usability criteria.

It does not guarantee success — but a well-run test should surface the key problems before your users hit them.



**Functionally correct
≠ usable**

Every button can work while users still cannot complete their task.



**Real users, real
tasks**

Watch representative users attempt genuine goals — don't ask your teammates.



**Observe, don't just
measure**

The “why” behind a failure matters as much as the success rate.



Findings → change

It isn't a test until the evidence changes the product.

REFERENCE

Jeffrey Rubin & Dana Chisnell — Handbook of Usability Testing (Wiley). The method below follows this classic.

Six required components

Rubin: it isn't a usability test unless all six are present.



1 • Specific objectives

Written problem statements & a test plan — what questions must this answer?



2 • Representative users

A real sample of the target population, recruited to a profile.



3 • Real environment

Represent the actual conditions of use, not an idealized default.



4 • Observation

Watch users work and review the product as they go.



5 • Quant + qual data

Collect measurements and impressions together.



6 • Analysis & recommendations

Turn observations into prioritized fixes.

Ways to run a usability test

Choose along three axes to fit budget, stage and question.



Moderated

Facilitator present; probes “why?”; rich qualitative insight — slower, costlier.



Unmoderated

Self-paced remote; cheap and scalable — no follow-up questions.



Qualitative

Think-aloud, interviews, observation — explains behaviour.



Quantitative

Task metrics, benchmarks, SUS — measures behaviour.

LOCATION

In-lab · remote synchronous · remote asynchronous · guerrilla / hallway — each trades control for reach.

Four types across the lifecycle

Rubin's test types — different moments call for different tests.



Exploratory — early

On sketches & wireframes. Evaluates the basic concept with shallow tasks; informal, heavy participant interaction.



Assessment — mid

After concepts are fixed. Well-defined tasks; less monitor interaction; quantitative measures collected.



Validation — late

Near release, “disaster insurance”: whole product incl. help & docs, against predetermined benchmarks.



Comparison — any time

Compare alternative designs with objective measures; combine the best features of each.

Test environments

From a single room to a purpose-built lab — separate observers from the user.



Simple single room

Observer sits near or removed from the evaluator — cheapest setup.



Electronic observation room

One-way glass / cameras feed a separate observation room.



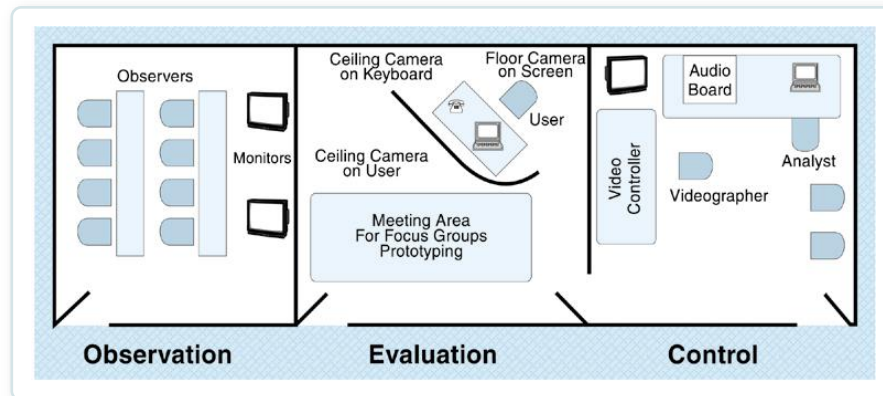
Classic usability lab

Dedicated test + observation + control rooms with recording rigs.

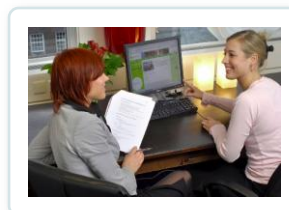


Mobile lab

Portable kit taken to the users' real environment.



Observation · Evaluation · Control zones

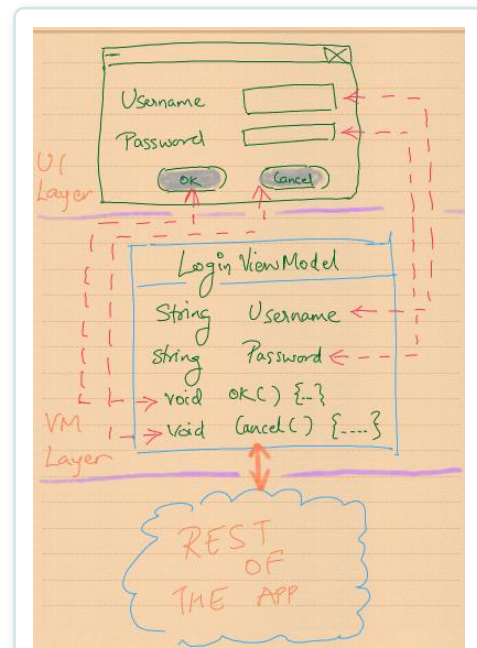


A usability session in progress

The 9-part test plan

Plan before you recruit — the plan is the contract for the study.

1. Purpose
2. Problem statement (specific questions)
3. Test objectives & tasks
4. User profile
5. Method & test design
6. Environment / equipment
7. Test-monitor role
8. Evaluation measures & data to collect
9. Report contents



Tasks come from a user model, not the code

Selecting tasks

Tasks are what users want to do — user view, not implementation. Specific & measurable.

Good tasks...

- are real goals: “create and file a document”, “find the right document”
- don’t tell the user how — expose flaws indirectly
- cover the key, frequent journeys
- have a defined success criterion & benchmark

EVALUATORS representative of the target users · independent-groups or within-subject design (avoid repeat-exposure bias) · adequate numbers · motivate/reward.

TASK-COMPONENT EXAMPLE

Task	Load paper into the copier
State	Paper tray empty
Success	Paper properly loaded
Benchmark	Done within 1 minute

2nd example: create a VM with the VMware wizard · state = software just loaded · success = working VM · benchmark = 30 s.

What to measure — performance vs preference

Collect both, then link each back to a problem statement.

PERFORMANCE DATA (OBJECTIVE)

- Time per task
- % of tasks completed (with / without assistance)
- Time to access information
- Count of incorrect selections
- Number of errors
- Number of help accesses
- System response time

PREFERENCE DATA (SUBJECTIVE)

- Rankings & opinions via questionnaires
- Likert rating scales
- Semantic differentials
- Fill-in (open) questions
- Check-box questions
- Branching questions
- Free-form comments

Questionnaire formats 1 — rating scales

LIKERT SCALE

“I found the GUI easy to use.”

Strongly disagree (−2) Disagree (−1) Neutral (0) Agree (+1) Strongly agree (+2)

SEMANTIC DIFFERENTIAL

“The File → Open menu is...”

Simple 3 2 1 0 1 2 3 Complex

SCORING Anchor scales symmetrically so ratings can be averaged; keep wording neutral to avoid leading the participant.

Questionnaire formats 2 — open, check-box, branching



Fill-in (open)

“List 0–4 aspects you found particularly easy to use.” Captures unprompted, qualitative signal.



Check-box

“How often do you use spell-check?” ☐ Never ☐ Sometimes ☐ Often ☐ Always — quick categorical data.



Branching

“Did you use Advanced Search?” → if yes: “Boolean or Relevance mode?” Routes users to relevant follow-ups.

DESIGN TIP Always collect preference data in the context of the concrete task the user just attempted — not in the abstract.

Summarizing the results

PERFORMANCE

- Completion time: mean, median, range, standard deviation
- System response-time statistics
- Task accuracy: % completing within time / regardless of time / with assistance
- Average error rate per task

PREFERENCE

- Counts & % per answer choice
- Average Likert scores
- Categorize free-form answers (positive / negative)
- Group similar free comments into themes

Analysis — from data to recommendations



Focus on failures

Start with failed tasks; identify each user error and its source.



Prioritize

Criticality = severity × probability.
Fix high-criticality issues first.



Compare groups

Analyse differences between user groups / conditions.



Recommend

End every study with concrete, prioritized recommendations.

LINKING EXAMPLE — “IS THE TUTORIAL EFFECTIVE?”

Task: Group A imports an image without the tutorial; Group B with it (an A/B design). **Performance:** compare error rates of the two groups. **Preference:** Likert rating + free comments on the tutorial.

What to probe for preference

Always ask these in the context of a concrete task.



Layout & information

Does the layout match the task? Is there an adequate amount of information — not too much, not too little?



Text & colors

Good headlines, help and warnings? Good, accessible colours?



Grouping & navigation

Related info grouped? Do users ever get lost? Is it organized by user tasks?



Icons & consistency

Are icons self-explanatory? Is behaviour consistent across screens?

TOOLS FROM THE COURSE

Usability testing tools

The tool survey from the original deck — screenshots preserved. A closing slide maps them to today's equivalents.

Remote & unmoderated testing

Recruit participants and collect task metrics without a facilitator.

The screenshot shows the Loop11 dashboard. At the top, there's a 'User Test' section with a 'Create New User Test' button. Below it, a 'Hide website preview' link and a 'Preview' button are visible. The 'Preview' button is next to a text input field containing 'http://www.fit.hcmus.edu.vn'. To the right, there are 'COMMON QUESTIONS' and a 'BUY TEST CREDITS' section. The 'Projects Under Construction' section lists projects with columns for PROJECT, CREATED, TASKS, EDIT, and DELETE. The 'Launched Projects' section lists projects with columns for PROJECT, LAUNCHED, RESPONSES, STATUS, REVIEW, ANALYSE, and DELETE. A 'View full list' link is present at the bottom right.

COMMON QUESTIONS

- If I require help and submit a query via email how long will it take to get a response?
- Can I provide feedback about Loop11?
- Can I speak to someone at Loop11 if I need assistance?
- Is credit card payment the only method of purchase?

BUY TEST CREDITS

Pre-purchase credits instead of paying as you go.
[Go to purchase page](#)

Projects Under Construction

PROJECT	CREATED	TASKS	EDIT	DELETE
View full list				

Launched Projects

PROJECT	LAUNCHED	RESPONSES	STATUS	REVIEW	ANALYSE	DELETE
SAMPLE PROJECT - Yahoo!	13 Apr 2009	224	Closed	View design	View report	Delete
View full list						

Loop11 — remote unmoderated tests, quantitative metrics

The screenshot shows the FiveSecondTest dashboard. At the top, there's a 'Create a new test' section. Below it, there's a 'Pick a test option' section with a table showing different test levels: free, bronze, silver, and gold. The 'free' option is selected. Below the table, there's an 'Upload your design' section with a text input field and a 'Upload' button.

fivesecondtest

Create a new test

1 Pick a test option

Choose the level of test you need based on your required number of responses, priority and features.

	free	bronze	silver	gold
results	5	31	57	84
priority	1	10	10	10
custom instruction	1	1	1	1
extra feedback	1	1	1	1
price	free! (0 tokens)	US\$ 5 (1 token)	US\$ 10 (2 tokens)	US\$ 15 (3 tokens)

2 Upload your design

Upload a mockup or screenshot of the design you want to test.

FiveSecondTest — first-impression testing

Recorded user sessions

Watch real users use the product and narrate their thoughts.



HomeFAQBuzzAbout UsTry It

Usability Testing Has Never Been Easier

The fastest, cheapest way to *find out why users leave your website*

For \$39 you get:

- Video of a visitor speaking their thoughts as they use your site
- Written summary describing the problems they encountered

“Use it and your site will get better.”
Evan Williams |  Co-Founder [More »](#)

Try It Risk-Free
1-Year Money-Back Guarantee



Quick Tour
UserTesting.com provides on-demand usability testing

UserTesting.com — on-demand recorded user tests

Welcome, lamquangvu.temp@gmail.com. [log out](#)

[Install](#) [Affiliate Program](#) free plan, 10 captures remaining this month, [upgrade](#) for more.

Welcome to userfly :). You have 10 user recordings this month, so let's get you up and running.

Add the following inside the `<head>` tag of your site:

```
<script type="text/javascript">
var userflyHost = (("https:" == document.location.protocol)
? "https://secure.userfly.com" :
"http://asset.userfly.com");
document.write(unescape("%3Cscript src='"+ userflyHost +
"/users/47609/userfly.js' type='text/javascript'%3E%3C"))
```

1

If you have a blog, install userfly with one click:

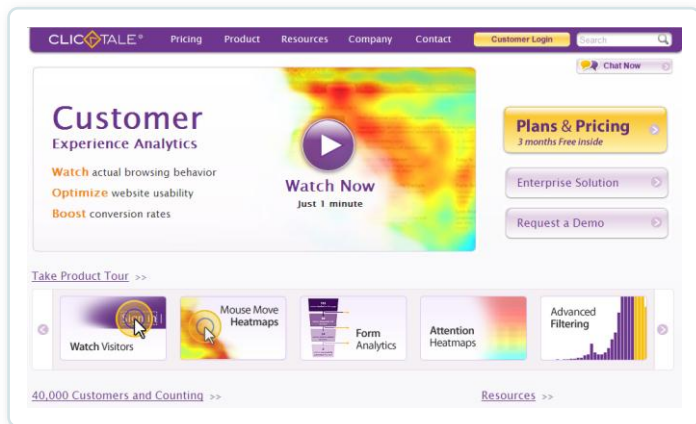
2 Visit your site to record your first user session.

3 [Click here](#) to watch your first recording!

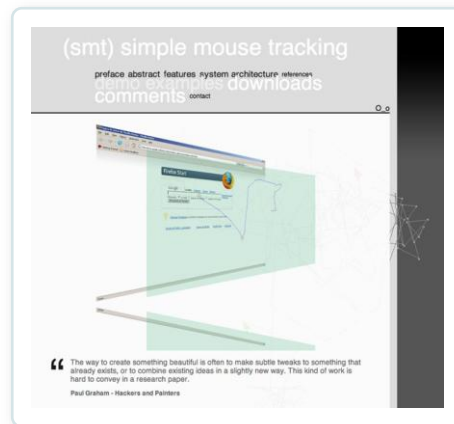
Userfly — records mouse & clicks with one line of code

Behavioural analytics & heatmaps

See where users click, scroll, hesitate and drop off — at scale.



ClickTale — click/scroll recording & heatmaps



Simple Mouse Tracking (open source)

WHAT THEY REVEAL Scroll & click heatmaps · form analytics (which fields cause abandonment) · session replays of individual visits.

Moderated capture & the comparison table

Clearleft presents

Silverback

Spontaneous, unobtrusive usability testing software for designers and developers.

Silverback makes it easy, quick and cheap for everyone to perform guerrilla usability tests with no setup and no expense, using hardware already in your Mac.

so says [Softrpedia](#) | [Next testimonial](#)



- Capture screen activity
- Video the participant's reactions
- Record the participant's voice
- Add chapter markers on the fly
- Control recording with the remote
- Export to Quicktime

Download
FREE FOR 30 DAYS

Purchase \$49.95
10% OF ALL PROFITS GO TO SAVE THE GORILLAS

Silverback — guerrilla usability recording (screen + webcam)

Tools	Web hay mobile app?	Main features					Free plan
		Session recordings	Heatmaps	Surveys	Funnels	Others	
hotjar	web	✓	✓	✓		Real-time feedback	Free
crazyegg	web	✓	✓	✓		A/B testing	Free trial
Microsoft Clarity	web	✓	✓			Insights, Google Analytics integration	Free
mouseflow	web	✓	✓		✓	User feedback	Free
fullstory	both	✓			✓	Conversions, Journey mapping	Free trial
smartlook	both	✓	✓		✓	Event tracking	Free trial

Course comparison: Hotjar · CrazyEgg · Clarity · Mouseflow · FullStory · Smartlook

Accessibility & colour-blindness checks

Part of usability: can everyone perceive the interface?

Colour Blindness Simulator
These are the results of your colour blindness simulation.

Original
This is the image you uploaded.



Simulation
This is how your image may appear to a person with protanopia.



Protanopia is a more common form of colour blindness, occurring in between 1% and 5% of males (varying by race) and in approximately 0.1% of females. You may like to simulate how someone with [deuteranopia](#) or [tritanopia](#) views your image.

Colour-blindness simulator (Etre)

Colorblind Web Page Filter
What are color blind anomalies? ⓘ

Please indicate a resource to be viewed, and a color filter to be applied to that resource.

Type in a URL:
<http://www.fit.hcmus.edu.vn>

And then pick a color filter:
Protanopia

FETCH AND FILTER



Colorblind Web Page Filter (Toptal ColorFilter)

REMEMBER ≈8% of men have some colour-vision deficiency — never rely on colour alone to convey meaning.

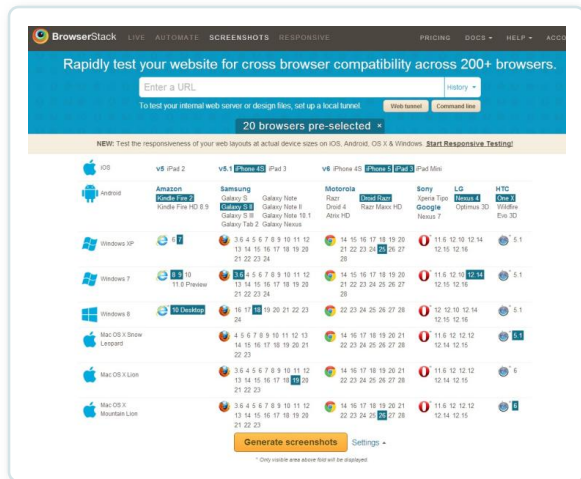
RELATED WEB-QA TOOLING

Compatibility & link checking

Two more tool families from the deck — the GUI must render everywhere, and every link must resolve.

Cross-browser & device testing

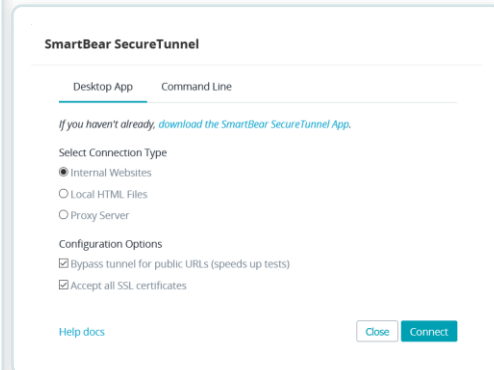
Run one test across thousands of real browser × OS × device combinations.



BrowserStack — 3000+ real combos



CrossBrowserTesting (SmartBear)



SecureTunnel — test internal builds

Link checkers

Crawl the site and classify every link: working / unverified / broken.

Link Checker
Enjoying this extension?

URL: <https://www.linkresearchtools.com/>

1 Total Links **165 (165/0)** **Export** **7** **Completed**

2 Working Links **75** **F/NF** **3** Unverified Links **90** **F/NF** **4** Broken Links **0** **F/NF**

5

Category	Working	Unverified	Broken
External	7 / 0	0 / 0	0 / 0
Same Root Domain	9 / 0	0 / 0	0 / 0
Internal	59 / 0	90 / 0	0 / 0

[More about Power*Trust](#) [Try LinkResearchTools](#)

LinkResearchTools — totals: working / unverified / broken

Links - Checked: 101 Remaining: 0

URL	Tip	Status	Protocol	Content	Size	In	Out	Last Modified
http://www.link-checker-pro.com/	Link Checker Pro - The leading...	OK	http	text/html	6707	0	17	05-Apr-04 12:14:48
http://www.link-checker-pro.com/index.htm	Link Checker Pro - The leading...	OK	http	text/html	6707	17	17	05-Apr-04 12:14:52
http://www.link-checker-pro.com/features.htm	Link Checker Pro - The leading...	OK	http	text/html	17980	17	13	05-Apr-04 12:14:56
http://www.link-checker-pro.com/screenshots.htm	Link Checker Pro - The leading...	OK	http	text/html	5580	17	17	05-Apr-04 12:15:00
http://www.link-checker-pro.com/download.htm	Link Checker Pro - The leading...	OK	http	text/html	6708	17	15	05-Apr-04 12:14:57
http://www.link-checker-pro.com/pricing.htm	Link Checker Pro - The leading...	OK	http	text/html	21230	17	23	05-Apr-04 12:15:00
http://www.link-checker-pro.com/contact.htm	Link Checker Pro - The leading...	OK	http	text/html	22440	17	23	05-Apr-04 12:14:56
http://www.link-checker-pro.com/support.htm	Link Checker Pro - The leading...	OK	http	text/html	6982	17	14	05-Apr-04 12:15:02
http://www.link-checker-pro.com/resellers.htm	Link Checker Pro - The leading...	OK	http	text/html	12616	17	14	05-Apr-04 12:14:59
http://www.link-checker-pro.com/links.htm	Link Checker Pro - The leading...	OK	http	text/html	8346	17	14	05-Apr-04 12:15:10
http://www.link-checker-pro.com/screenshots_2.htm	Link Checker Pro - The leading...	OK	http	text/html	12528	17	74	05-Apr-04 12:15:13
http://www.link-checker-pro.com/special_offer.htm	Link Checker Pro - The leading...	OK	http	text/html	8172	17	18	05-Apr-04 12:15:07
http://www.link-checker-pro.com/images/logo_40.gif	Link Checker Pro - The leading...	OK	http	image/gif	834	17	14	05-Apr-04 12:14:55
http://www.link-checker-pro.com/images/test_40.gif	Link Checker Pro - The leading...	OK	http	image/gif	4630	17	14	05-Apr-04 12:14:55
http://www.link-checker-pro.com/images/map_150.gif	Link Checker Pro - The leading...	OK	http	image/gif	22868	2	N/A	28-Dec-03 09:19:26
http://www.link-checker-pro.com/images/map_150.gif	Link Checker Pro - The leading...	OK	http	image/gif	9987	2	N/A	28-Dec-03 09:19:15
http://www.link-checker-pro.com/images/Man_12.gif	Link Checker Pro - The leading...	OK	http	image/gif	67084	1	N/A	28-Dec-03 09:17:55
http://www.link-checker-pro.com/images/Man_11.gif	Link Checker Pro - The leading...	OK	http	image/gif	69034	1	N/A	28-Dec-03 09:18:32
http://www.link-checker-pro.com/images/Man_14.gif	Link Checker Pro - The leading...	OK	http	image/gif	89597	1	N/A	28-Dec-03 09:19:07
http://www.link-checker-pro.com/links.zip	Link Checker Pro - The leading...	OK	http	application/zip	4213	1	N/A	04-Apr-04 13:21:02
http://download.com/3000-2181-1025354.htm	Link Checker Pro - Reviews and...	OK	http	text/html	10706	1	N/A	05-Apr-04 12:15:47
http://digidownload.com/2-2531-25322532.htm	Link Checker Pro - The leading...	OK	https	text/html	N/A	4	N/A	05-Apr-04 12:15:25
http://recurseshare.com/share/checkout.html?ps	Link Checker Pro - The leading...	OK	https	text/html	47319	2	N/A	05-Apr-04 12:15:57
http://recurseshare.com/share/checkout.html?ps	Link Checker Pro - The leading...	OK	https	text/html	47320	2	N/A	05-Apr-04 12:16:24
http://recurseshare.com/share/checkout.html?ps	Link Checker Pro - The leading...	OK	https	text/html	N/A	2	N/A	05-Apr-04 12:15:45
http://recurseshare.com/share/checkout.html?ps	Link Checker Pro - The leading...	OK	https	text/html	47312	2	N/A	05-Apr-04 12:16:57
http://recurseshare.com/share/checkout.html?ps	Link Checker Pro - The leading...	OK	https	text/html	N/A	2	N/A	05-Apr-04 12:16:10
http://recurseshare.com/share/checkout.html?ps	Link Checker Pro - The leading...	OK	https	text/html	47323	2	N/A	05-Apr-04 12:17:24
http://www.link-checker-pro.com/written_quote.htm	Link Checker Pro - The leading...	OK	http	text/html	10428	3	18	05-Apr-04 12:17:32
http://www.link-checker-pro.com/payment_methods.htm	Link Checker Pro - The leading...	OK	http	text/html	12610	4	17	05-Apr-04 12:15:42

OK: 101 M: 0 F: 0 P: 0 O: 0 U: 0

Xenu's Link Sleuth — desktop crawler

ALSO W3C Link Validator (validator.w3.org/checklink) · ahrefs · Dr. Link Check · Sitechecker · SE Ranking · plus HTML & CSS validators.

From the course tools to today's (2024–2026)

Many deck tools are dated — here's the current equivalent for each job.

Job	In the deck	Use today
Remote unmoderated	Loop11 · FiveSecondTest	Maze · Lyssna · Useberry (+ Loop11 still)
Recorded sessions	UserTesting · Userfly	UserTesting · Lookback (Userfly retired)
Heatmaps / replay	ClickTale	Microsoft Clarity (free) · Hotjar · FullStory
Comparison set	Hotjar/CrazyEgg/Mouseflow...	+ Clarity (free) · Contentsquare
Accessibility	Etre · Toptal ColorFilter	axe DevTools · WAVE · Lighthouse
AI usability (new)	—	Attention Insight · Maze AI · Synthetic Users

EXERCISE

Your turn: run a small usability study

Plan & tasks

Write a mini 9-part test plan for MentorUS. Define 3 real tasks, each with a success criterion and a benchmark time.

Run & measure

Do a 3-user think-aloud. Capture performance (time, errors, % complete) and preference (a short Likert + SEQ).

Analyse & tool up

Rank issues by severity $\times$ probability. Add a free tool (Clarity heatmap or axe-core) — what did it reveal that users didn't?

Deliverable: a 1-page report — key findings, prioritized recommendations, and one screenshot of evidence.

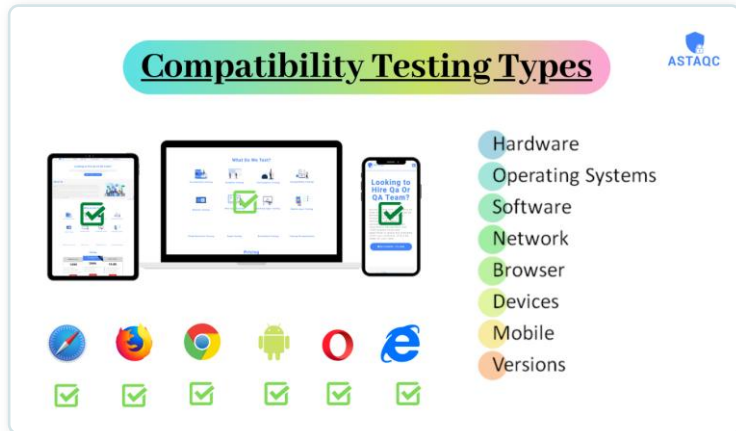
PART 3

Browser & Device Compatibility

One app, thousands of browser × OS × device combos — a tool spectrum from offline screenshots to online real-device simulation.

Why compatibility testing — and the matrix

The same code renders differently per engine, OS, device, viewport, version & network.



Compatibility dimensions (course diagram)

MINDSET the app “works on my machine” but breaks on a user’s browser — differences in layout, fonts, events & APIs.



Three rendering engines

Blink (Chrome/Edge/Opera) · WebKit (Safari & ALL iOS browsers) · Gecko (Firefox).



Mobile Safari diverges most

iOS forces WebKit — a WebKit bug can’t be reproduced on desktop Chrome. Always test it.



Build the matrix from analytics

Test the top ~10–15 combos your real users have — not everything.



Interop & Baseline

Interop 2025 hit ~97% cross-browser pass. If a feature is Baseline “Widely available”, ship it without fallbacks.

Emulator vs Simulator vs Real device

Accuracy rises left → right; cost and effort rise the same way.



Emulator

Mimics the device OS AND hardware/CPU at a low level (e.g. Android Emulator).
More accurate, heavier to run.



Simulator

Mimics the software environment only, on host resources (e.g. iOS Simulator — real WebKit, but no real GPU/sensors/cellular).



Real device

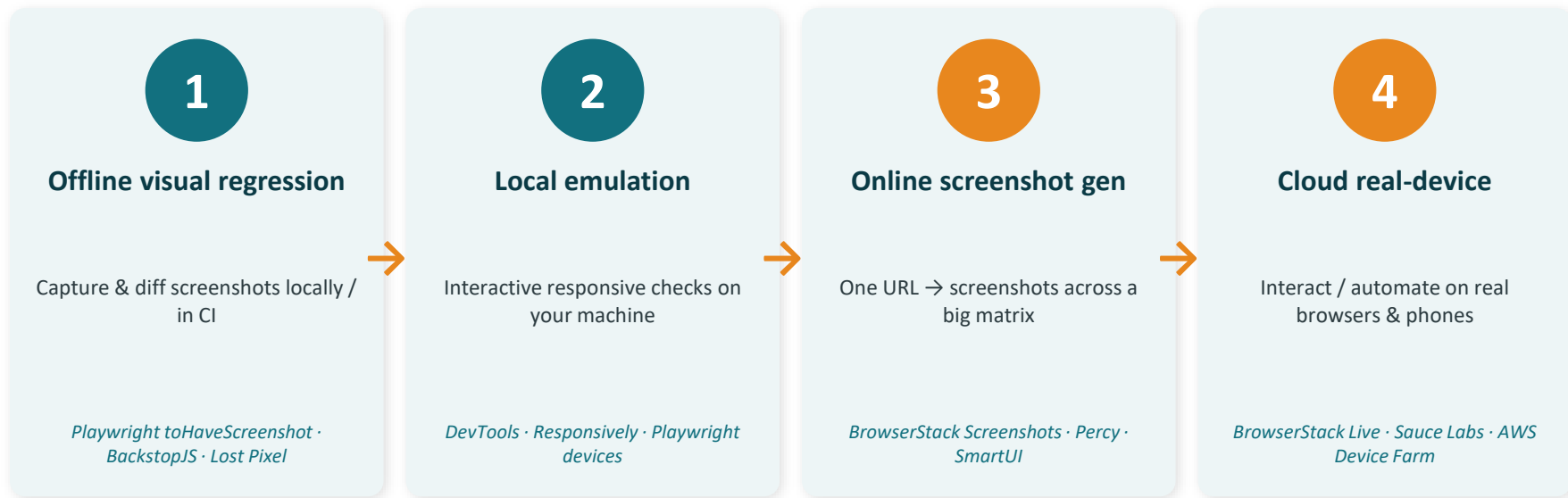
The physical phone = ground truth: real GPU, touch, notch, network, thermal throttling. Highest cost, lowest scale.

TRADEOFF

Emulators/simulators win on speed & CI scale; real devices win on fidelity. Cloud device farms give real devices without owning them.

The compatibility tool spectrum

Four rungs — from free & offline to real-device & online.



RULE Free & fast on the left for every commit; real-device & paid on the right before release. Use all four — they catch different bugs.

Rung 1 — offline screenshot capture & visual regression

Runs on your machine / CI, no cloud account. Catches CSS regressions automatically.

OFFLINE VISUAL CHECK — ZERO CLOUD COST

```
// Playwright – visual assertion, 3 engines
await expect(page)
  .toHaveScreenshot('home.png',
    { maxDiffPixelRatio: 0.01 });
// runs on chromium + firefox + webkit
```

How it works: capture a baseline → capture current → pixel/AI diff → fail on mismatch above a threshold.



BackstopJS

Mature OSS full-page visual regression; config-driven, HTML diff reports.



Lost Pixel / reg-suit

Modern OSS diffing; reg-suit stores baselines in S3 to keep repos light.



Catches / limits

Catches layout shifts, CSS & font regressions. Limit: pixel noise (anti-aliasing, darts) → flaky; only local engines.

EXAMPLE Playwright bundles Chromium, Firefox & WebKit — one toHaveScreenshot run diffs all three engines in CI, free.

Rung 2 — local emulation (interactive, on your machine)

Fast responsive checks at real breakpoints — but not always a real mobile engine.

ONE-LINE DEVICE EMULATION

```
// Playwright device descriptor
const ctx = await browser.newContext({
  ...devices['iPhone 14'] // viewport,
});                       // UA, DPR, touch
```

EXAMPLE Responsively App shows the page at 4 breakpoints side-by-side — click once, all panes react. Free & open source.

Great for the inner loop; confirm findings on a real engine (rung 4) before you trust them.



Chrome DevTools / FF RDM

Device toolbar, network throttling, touch — free, built in.



Responsively App (OSS, free)

Many viewports at once, synced — great for 360/768/1024/1440.



Android Emulator / iOS Simulator

Full Android OS / real WebKit Safari (iOS, macOS only).

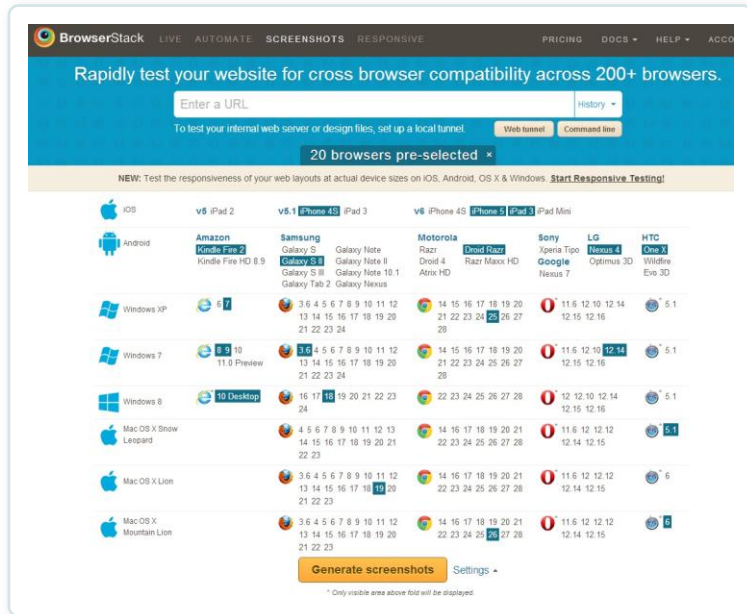


Limit

DevTools “iPhone” is still Blink; emulators lack real GPU & network.

Rung 3 — online screenshot generation

Point at a URL, pick environments, get screenshots back — fast fan-out, non-interactive.



BrowserStack — select browsers, then Generate screenshots



BrowserStack Screenshots

One URL → screenshots across a slice of 3,500+ real browser/OS combos in minutes.



Percy (by BrowserStack)

Visual testing in CI across responsive widths & browsers. Free plan: 5,000 screenshots/month.



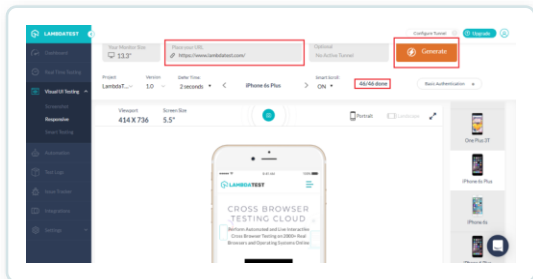
SmartUI (TestMu AI, ex-LambdaTest)

Batch screenshots + visual-diff engine with Layout Comparison (2025).

EXAMPLE Paste one URL → tick 20 browsers → Generate → 20 screenshots to eyeball the layout everywhere.

Rung 4 — cloud real-device: interact & simulate

Drive a real Safari/Chrome or a real phone — manually or via automation.



Live interactive session (real browser)

INTERACTIVE / AUTOMATED — WEB

- BrowserStack Live + Automate
- TestMu AI (ex-LambdaTest) + KaneAI
- Sauce Labs · SmartBear BitBar
- Tunnels: BrowserStack Local, SecureTunnel

REAL-DEVICE — MOBILE APPS

- BrowserStack App Live / App Automate
- AWS Device Farm (parallel real devices)
- Firebase Test Lab (free tier)
- Sauce Labs RDC · Kobiton · pCloudy

EXAMPLES Open a real iPhone 15 Safari and tap through checkout · run an Appium suite across 20 real devices in parallel on AWS Device Farm.

AI & the state of the art (2024–2026)

AI attacks the two bottlenecks: authoring $\times N$ configs, and reviewing $\times N$ screenshots.



Percy Visual Review Agent (2025)

AI boxes only meaningful changes with plain-English summaries; filters anti-aliasing noise — up to 3× faster review.



Applitools Eyes Visual AI

Layout match ignores content & flags structure; Dynamic match (2025) auto-suppresses dates/emails/card numbers.



TestMu AI — KaneAI + SmartUI

Natural-language / agentic test authoring across thousands of configs; SmartUI Layout Comparison.



Flaky-test detection

BrowserStack's AI flags flaky tests from execution history — the active research thread [here](#).

FREE STUDENT STACK

Playwright (3 engines + toHaveScreenshot) · BackstopJS / Lost Pixel · Responsively App · Percy free tier · cloud free trials.

Choosing the right rung

Each approach catches different bugs — layer them.

Approach	Example tools	Interact?	Real engine?	Best for
Offline visual regression	Playwright · BackstopJS	No	Local engines	CSS regressions (free)
Local emulation	DevTools · Responsively	Yes	Partial	Responsive checks
Online screenshot gen	BrowserStack · Percy	No	Yes	One URL, many envs
Cloud live web	BrowserStack Live · Sauce	Yes	Yes	Reproduce Safari bugs
Real-device mobile	AWS Device Farm · Firebase	Yes+auto	Ground truth	Native app fidelity